

THE 14-DAY BEAST MODE BLUEPRINT

JavaScript + DOM — Complete Mastery Execution Plan

Duration	14 Days (07 July - 20 July 2026)
Core Tracks	JavaScript (Phase 1-6) + DOM (15 Modules / 47 Topics)
Practice Banks	54-Task Sheet, Phase-6 DOM Sheet, Phase 1-5 JS Sheets, OOP Sheet, this/call/apply/bind/Prototypes/Classes Sheet
Capstone Projects	Task Manager FinTrack Pro Productivity Dashboard
Daily Commitment	Full Day (Morning - Night), Structured Blocks
End Goal	Interview-Ready + Project-Ready JS & DOM Foundation

“Coverage complete hone ka matlab mastery nahi hota — ye 14 din tumhara **foundation lock-in** hain.
Mastery uske baad, real projects aur real bugs se aayegi.”

HOW TO USE THIS PLAN

- Har din ka target **usi din** khatam karna hai — agla din shuru mat karo jab tak previous din ka Practice Task complete na ho.
- Practice Task kabhi skip mat karo — sirf notes/theory padhna revision nahi hai. Khud likh ke ya bol ke answer dena hi real prep hai.
- Jo bhi topic padho, usi din raat ko us concept ka ek chhota (20-30 min) hands-on code snippet zaroor likho.
- Mentor sirf concept samjhayega aur mistakes correct karega — coding khud karni hai. Pehle khud debug karne ki koshish karo, phir madad lo.
- Day 13-14 ke mock tests mein honest raho — jo weak lage sirf wahi topic Day 14 mein dobara touch karo, naya kuch mat seekho.
- Overlapping practice questions (jaise Counter, Todo List, Dark Mode jo multiple sheets mein hain) — dusri baar bina reference dekhe khud se banao. Ye spaced repetition hai, real retention isi se aata hai.

AFTER DAY 14 — WHAT HAPPENS NEXT

- Day 15-17: Mock interviews (JS+DOM theory + 2-3 live coding questions), fix only the specific gaps found.
- Build the 3 capstone projects fully (Task Manager, FinTrack Pro, Productivity Dashboard) if not already finished during the plan.
- Start applying for roles in parallel — don't wait for '100% perfect'.
- Move to React (foundation will make this ~60% faster), then Next.js.
- Parallel track for 'good package' product companies: start DSA + basic Frontend System Design alongside React.

DAY 1 | 07 Jul (Mon) | Engine Foundations & DOM Basics

PART 1 — JavaScript	Phase 1 — Foundations: Variables (var/let/const), data types (primitive vs non-primitive), typeof, type conversion vs coercion, all operators, string & number basics, conditionals (if/else/switch), loops (for/while/do-while/for-of/for-in), break/continue.
PART 2 — DOM	Module 01 + 02: What is DOM (internal parsing process), DOM vs HTML vs BOM, Node Types, window vs document, Parent/Child/Sibling relationships, Child vs Descendant, Live Tree concept (with the live HTMLCollection loop-bug demo).
PART 3 — Practice / Production	JS: Solve Phase-1 Notion sheet (20-05-26) fully. Write 10 quick output-prediction snippets for coercion (==, ===) yourself. DOM: Self-answer every Q&A in Module 01 & 02 out loud, no notes. Open DevTools console and verify window.document === document.

Goal: Comfortable writing basic JS + rock-solid DOM vocabulary.

DAY 2 | 08 Jul (Tue) | Functions + CSSOM/Render Tree

PART 1 — JavaScript	Phase 2 — Functions: declaration vs expression, arrow functions (full+shorthand), parameters vs arguments, default/rest params, return values, first-class functions, IIFE, callbacks (intro), higher-order functions (intro), pure vs impure, recursion basics.
PART 2 — DOM	Module 03 + 04: CSSOM construction & why it's render-blocking, Specificity/Cascade/Inheritance, JS↔CSSOM interaction + layout thrashing intro, Render Tree construction, display:none vs visibility:hidden vs opacity:0, Layout & Paint phases, full pixel pipeline.
PART 3 — Practice / Production	JS: Write 5 small functions (factorial, palindrome, reverse string, sum array manually, one recursive function). Task 1-5 from the 54-Task sheet (Easy JS). DOM: Explain the full Render Tree pipeline out loud without notes; test opacity:0 vs visibility:hidden click-ability yourself in browser.

Goal: Never confuse Layout-triggering vs Paint-only vs Composite-only CSS changes again.

DAY 3 | 09 Jul (Wed) | Arrays, Objects + DOM Selection

PART 1 — JavaScript	Phase 2 — Arrays & Objects: all core array methods (push/pop/slice/splice/etc), iteration methods (map/filter/reduce/find/some/every), array destructuring, spread/rest, object literals, dot vs bracket access, object destructuring, Object.keys/values/entries, Object.freeze/seal.
PART 2 — DOM	Module 05: getElementById vs querySelector performance, HTMLCollection vs NodeList (live vs static, WHY internally), the live-collection removal bug + fix, closest() vs querySelector direction, right-to-left selector matching.
PART 3 — Practice / Production	JS: Task 6 (Product Data Processor - map/filter/reduce) from 54-Task sheet. Phase-2 Sheet-1 Notion (23-05-26). DOM: Practice every selection method live in browser console on any real webpage; time getElementById vs querySelector('#id') mentally.

Goal: map/filter/reduce should feel automatic, not memorized.

DAY 4 | 10 Jul (Thu) | Execution Context/Hoisting/Scope + Traversal

PART 1 — JavaScript	Phase 3 — How JS Works: Engines overview (V8 etc), Execution Context (GEC/FEC), Creation vs Execution phase, Call Stack + stack overflow, var/let/const hoisting + TDZ, function declarations vs expressions hoisting, Global/Function/Block/Lexical scope, Scope Chain.
PART 2 — DOM	Module 06: Node-based vs Element-based traversal (parentNode vs parentElement edge case with document), closest() self-matching trap, nextSibling vs nextElementSibling whitespace trap, DFS traversal of DOM subtree (recursive + iterative), TreeWalker/NodeIterator.
PART 3 — Practice / Production	JS: 15 hoisting/scope output-prediction questions (write your own + solve Phase-2 Sheet-2 Notion 26-05-26). DOM: Traverse a nested HTML block in console using every traversal property; write the recursive traverseDOM() function from scratch.

Goal: Predict ANY hoisting/scope output correctly, no guessing.

DAY 5 | 11 Jul (Fri) | Closures (Deep) + DOM Manipulation

PART 1 — JavaScript	Phase 3 — Closures: real definition, step-by-step trace, private data/function factories/ counters/currying use-cases, classic closure+loop problem (var vs let), common interview questions. <i>Homework:</i> "How JS Works" blog/documentation task.
PART 2 — DOM	Module 07: innerHTML vs textContent vs innerText (security+performance), createElement+ appendChild vs insertAdjacentHTML, DocumentFragment (why + reflow demo), cloneNode deep vs shallow + event listener NOT cloned trap, replaceChild/replaceWith/replaceChildren, appendChild MOVES not copies.
PART 3 — Practice / Production	JS: Task 4, 7, 11 from 54-Task sheet (Private Counter, Debounce, Memoization — all closures). Phase-3 Notion sheet (31-05-26). DOM: Build a dynamic list using createElement + DocumentFragment (1000 items, compare reflow feel with/without fragment).

Goal: Closures should stop feeling like 'magic' — you can explain the trace step by step.

DAY 6 | 12 Jul (Sat) | this/call/apply/bind + Attributes & Styles

PART 1 — JavaScript	Phase 4 — this: this in global/regular functions/methods/arrow functions/event handlers/ setTimeout/strict mode. call, apply, bind explicit binding.
PART 2 — DOM	Module 08: Attribute vs Property (input.value vs getAttribute('value') divergence), custom attributes NOT auto-reflected, classList methods incl. toggle(force) & replace(), dataset API + camelCase conversion, element.style vs getComputedStyle(), !important overriding inline JS styles + setProperty() fix.
PART 3 — Practice / Production	JS: Practice Sheet (this/call/apply/bind/Prototypes/Classes, 15-06-26) — this section only. 10 this-context output-prediction questions. DOM: Build a theme switcher using classList.toggle('dark', isDark) pattern; test the !important override bug yourself.

Goal: Never get confused by `this` inside a callback again.

DAY 7 | 13 Jul (Sun) | Prototypes/Classes + Events Basics

PART 1 — JavaScript	Phase 4 — Prototypes & Classes: <code>__proto__</code> vs prototype, prototype chain, <code>Object.create</code> , constructor functions + <code>`new`</code> (4 steps), ES6 class syntax, <code>extends/super</code> , static methods, getters/setters, private fields (<code>#</code>), "classes are sugar over prototypes" proof.
PART 2 — DOM	Module 09: <code>addEventListener</code> 3 params (capture/once/passive), <code>event.target</code> vs <code>currentTarget</code> , <code>preventDefault</code> vs <code>stopPropagation</code> vs <code>stopImmediatePropagation</code> , <code>passive:true</code> for scroll perf, input vs change event difference, <code>DOMContentLoaded</code> vs <code>window.onload</code> .
PART 3 — Practice / Production	JS: OOP Practice sheet (13-06-26) fully + this/prototypes/classes sheet (remaining part). Build one constructor-function AND one ES6-class version of the same object. DOM: Task 10 (Mini Event Emitter) from 54-Task sheet; test <code>target</code> vs <code>currentTarget</code> with a nested click.

Goal: Explain 'classes are syntactic sugar' with a live code proof, not just the phrase.

DAY 8 | 14 Jul (Mon) | Promises + Events Advanced

PART 1 — JavaScript	Phase 5 — Promises: sync vs async need, callbacks + callback hell, Promise states (pending/fulfilled/rejected), <code>new Promise</code> , <code>.then/.catch/.finally</code> , chaining + error propagation, <code>Promise.all/race/allSettled/any</code> .
PART 2 — DOM	Module 10: Full 3-phase event flow (Capture→Target→Bubble) with nested example, why Capturing exists (<code>stopPropagation</code> -proof use case), Event Delegation full depth (memory + dynamic content benefit), Custom Events (<code>CustomEvent</code> + detail + <code>bubbles:true</code> trap).
PART 3 — Practice / Production	JS: Task 8 (Sequential Task Runner with Promises) + Phase-5 Drive sheet (02-06-26). DOM: Task 48 (Decoupled Cart via Custom Events) + Task 31/34/35 (delegation-based) from 54-Task sheet.

Goal: Explain event delegation's memory benefit AND dynamic-content benefit, not just 'it's convenient'.

DAY 9 | 15 Jul (Tue) | Async/Await + Forms

PART 1 — JavaScript	Phase 5 — async/await: syntax+rules, try/catch error handling, sequential vs parallel execution, common mistakes (forgetting await/try-catch).
PART 2 — DOM	Module 11: document.forms named access, Constraint Validation API (checkValidity, reportValidity, ValidityState, setCustomValidity + the 'forgot to reset' bug), novalidate/formnovalidate, FormData API (why better than manual .value reading, file uploads), disabled vs readonly submission trap.
PART 3 — Practice / Production	JS: Rewrite Task 8's Promise chain using async/await. Solve Practice session06 (07-06-26). DOM: Task 26 (Required Field Errors) + Task 9 (API Data Cleaner, fetch+try/catch) from 54-Task sheet; build a form using novalidate + custom ValidityState-driven error messages.

Goal: Never forget an await or a try/catch again — make it muscle memory.

DAY 10 | 16 Jul (Wed) | Event Loop + Fetch + DOM Performance

PART 1 — JavaScript	Phase 5 — Event Loop: call stack revisited, Web/Browser APIs, macrotask (callback) queue, microtask queue (Promises/queueMicrotask), exact scheduling order, tricky output-prediction problems. Fetch API + HTTP calls, JSON.parse/stringify.
PART 2 — DOM	Module 12: Reflow vs Repaint vs Composite cost ranking + why reflow cascades, Critical Rendering Path optimization (defer vs async exact difference), Debounce vs Throttle (implement both from scratch + decision table), requestAnimationFrame vs setTimeout, Layers & will-change (+ overuse warning).
PART 3 — Practice / Production	JS: 5 event-loop output-prediction puzzles (self-written). Task 9's fetch exercise using JSONPlaceholder. DOM: Task 7 (Debounce Utility) already done Day5-equivalent — now implement Throttle from scratch too; Task 47 (Debounced Live API Search) from 54-Task sheet.

Goal: Predict ANY setTimeout/Promise output-order puzzle correctly, every time.

DAY 11 | 17 Jul (Thu) | Modern ES6+ + Virtual DOM

PART 1 — JavaScript	Phase 6 — ES6+ deep dive: advanced destructuring, advanced spread/rest, tagged templates, optional chaining (?.), nullish coalescing (??), short-circuit patterns, logical assignment operators (??=, =, &&=), ES Modules (import/export), brief CommonJS, dynamic imports.
PART 2 — DOM	Module 13: Virtual DOM — the REAL problem it solves (not just 'faster'), Reconciliation/ Diffing algorithm ($O(n)$ heuristic, why not $O(n^3)$), why `key` prop matters (concrete reorder bug), when VDOM is NOT faster (clock counter-example), React Fiber (interruptible/priority scheduling).
PART 3 — Practice / Production	JS: Refactor 3 old snippets (Day 1-5) using optional chaining/nullish coalescing/destructuring. DOM: Explain the Virtual DOM diffing + key-prop bug out loud with a concrete example, no notes.

Goal: When you eventually learn React, none of this should feel new — only the syntax will.

DAY 12 | 18 Jul (Fri) | Error Handling/Map-Set + Browser APIs

PART 1 — JavaScript	Phase 6 — Error Handling & Built-ins: try/catch/finally, throw, built-in error types (TypeError/ReferenceError etc), custom error classes, Map & Set (+ WeakMap/WeakSet briefly), Date object, Regex basics, Iterators/Generators (intro).
PART 2 — DOM	Module 14: Shadow DOM (open vs closed, true CSS encapsulation), MutationObserver (async+batched, replaced old sync Mutation Events), IntersectionObserver (replaces expensive scroll+ getBoundingClientRect polling), ResizeObserver (vs window.resize gap), side-by-side comparison of all 3 observers.
PART 3 — Practice / Production	JS: Write a custom Error class + try/catch demo using it. Practice session questions (08-06-26). DOM: Task 52 (Lazy-Loaded Image Gallery using IntersectionObserver) + Task 45 (Infinite Scrolling List) from 54-Task sheet.

Goal: Map correct real-world scenario to correct Observer API without hesitation.

DAY 13 | 19 Jul (Sat) | Storage/Best Practices + Memory + FULL MOCKS

PART 1 — JavaScript	Phase 6 — Storage & Best Practices: localStorage/sessionStorage, cookies (brief), clean code/naming/debugging with DevTools, reading error messages. FULL JS MOCK: 30 mixed questions across ALL 6 phases (self-test, spoken answers).
PART 2 — DOM	Module 15: Detached DOM node leaks (concrete example), event-listener+closure retention leak mechanism, how frameworks prevent (and still allow) leaks, WeakMap for DOM metadata, final consolidated performance checklist (10 points). FULL DOM MOCK: 20 mixed questions across ALL 15 modules.
PART 3 — Practice / Production	Task 54 (localStorage vs sessionStorage Visit Tracker) + Task 42/43 (Persistent Todo, Remembered Theme) from 54-Task sheet. Today's real task: run both mocks HONESTLY. Write down every topic you hesitated on — this list becomes tomorrow's ONLY revision list.

Goal: Know EXACTLY which specific sub-topics are still weak, with zero self-flattery.

DAY 14 | 20 Jul (Sun) | FINAL: Combined Mock + Weak-Spot Repair + Capstone Kickoff

PART 1 — JavaScript	Re-revise ONLY the weak topics flagged from Day 13's JS mock. No new topics today.
PART 2 — DOM	Re-revise ONLY the weak topics flagged from Day 13's DOM mock. No new topics today.
PART 3 — Practice / Production	Full combined mock interview: JS + DOM mixed, timed, answers spoken out loud (record yourself if possible). Then start Task-Manager Using DOM (first capstone project) — this combines Modules 07/09/10/15 (manipulation, events, delegation, state-driven rendering) directly. Continue FinTrack Pro and Productivity Dashboard over the following days as the true integration test of all 14 days combined.

Goal: Walk out of Day 14 able to build, debug, and explain — not just recite.

DAY 14 COMPLETE.

Ye 'mastery' nahi, ye **solid, gap-free, interview-ready foundation** hai — jo abhi tumhe bahut logo se aage rakhta hai. Ab building continue karo: bade projects, React, Next.js, aur real bugs — wahi asli mastery ka raasta hai.

Beast mode: activated.